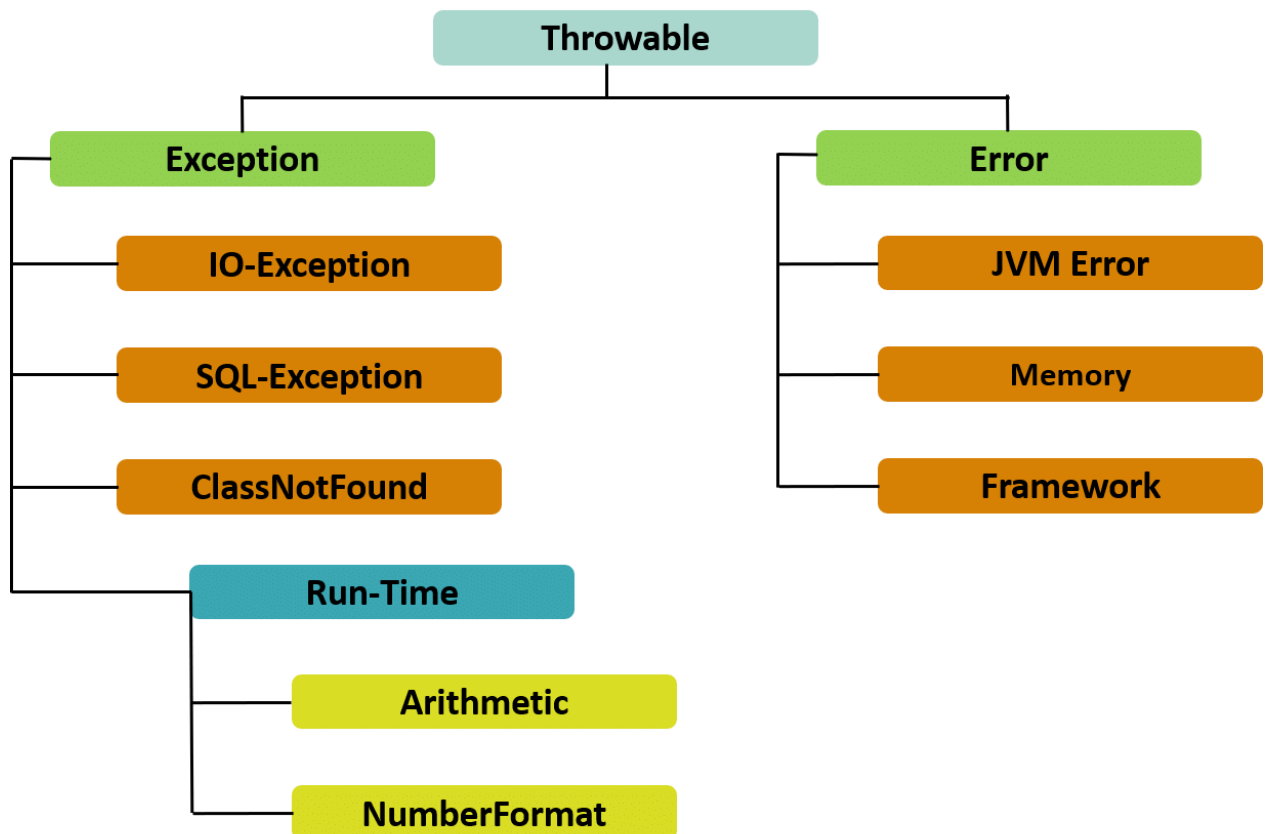


Exception Handling

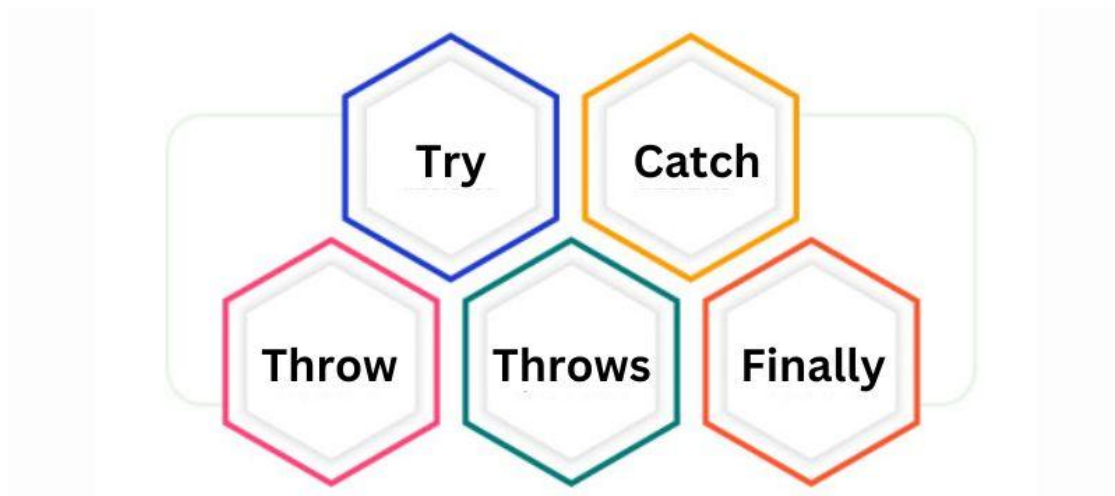
Exception Hierarchy :-

- In Java, all exceptions and errors are derived from the Throwable class.
- The Throwable class has two main subclasses: Exception and Error.
- The Exception class represents conditions that a program can catch and handle.
- Exceptions are divided into two types -
 - Checked Exceptions – Checked at compile time.
Examples –
 1. IOException
 2. SQLException
 3. ClassNotFoundException
 - Unchecked Exceptions – Checked at runtime.
Examples –
 1. NullPointerException
 2. ArithmeticException
 3. ArrayOutOfBoundsException
- The Error class represents serious problems that applications should not try to handle (e.g., OutOfMemoryError, StackOverflowError).
- This hierarchy helps organize different types of runtime problems in a structured way.



Keywords in Exception Handling :-

- Java exception handling uses five keywords such as try, catch, throw and throws and finally.



1) try

- The “try” keyword is used to specify the exception block.

Syntax –

```
try {  
    // Code that may cause an exception  
}
```

2) catch

- The “catch” keyword is used to specify the solution.

Syntax –

```
catch (ExceptionType e) {  
    // Code to handle the exception  
}
```

3) finally

- The “finally” keyword has a mandatorily executable code.

Syntax –

```
finally {  
    // Code that always executes  
}
```

Example –

```
package exception_handling;

public class finally_demo {

    public static void main(String[] args) {
        try
        {
            int x=4/0;
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
        finally
        {
            System.out.println("Finally Block Executed");
        }
        System.out.println("Bye");
    }
}
```

Output –

```
java.lang.ArithmeticException: / by zero
Finally Block Executed
Bye
```

4) throw

- The “throw” keyword throws an exception.

Syntax –

```
throw new ExceptionType("Error message");
```

5) throws

- The “throws” keyword is used to declare an exception.

Syntax –

```
void myMethod() throws ExceptionType {
    // Code
}
```

Customized Exception :-

- A customized exception is created by the programmer.
- It extends Exception (checked) or RuntimeException (unchecked).
- It is used to handle specific application errors.

How to Create Customized Exception :-

1. Create a class that extends Exception or RuntimeException.
2. Define a constructor to pass a custom error message.
3. Use the throw keyword to throw the exception.
4. Call the exception from a method or block where needed.
5. Handle it using a try-catch block.

```
// AgeException.java  
package pack;  
  
public class AgeException extends Exception  
{  
    public AgeException(String s)  
    {  
        super(s);  
    }  
}
```

Tutorial4us.com

Example –

```
package exception_handling;  
  
public class InvalidAgeException extends Exception {  
    public InvalidAgeException(String msg)  
    {  
        super(msg);  
    }  
}  
  
import java.util.Scanner;  
  
public class throw_keyword {  
    public static void main(String[] args) throws InvalidAgeException {
```

```
Scanner sc=new Scanner(System.in);
System.out.println("Enter the Age: ");
int age=sc.nextInt();

if(age>=18)
{
    System.out.println("Eligible for Voting");
}
else
{
    throw new InvalidAgeException("Not Eligible for Voting");
}

}
```

Output –

Enter the Age:

15

Exception in thread "main" [exception handling.InvalidAgeException](#): Not Eligible for Voting
at [exception_handling.throw_keyword.main\(throw_keyword.java:18\)](#)

Task 1 – InvalidAadharNumberException

```
package exception_handling;

public class InvalidAadharNumberException extends Exception{
    public InvalidAadharNumberException(String msg)
    {
        super(msg);
    }
}

package exception_handling;

import java.util.Scanner;

public class Aadhar {

    public static void main(String[] args) throws InvalidAadharNumberException {
        Scanner sc=new Scanner(System.in);
        System.out.println("Enter the Aadhar Number:");
        String aadhar_num=sc.next();

        if(aadhar_num.length() !=12 || !aadhar_num.matches("[0-9]+"))
        {
            throw new InvalidAadharNumberException("Invalid Aadhar Number");
        }
        else
        {
            System.out.println("Valid Aadhar Number: " + aadhar_num);
        }
    }
}
```

Output –

Enter the Aadhar Number:

234567890

Exception in thread "main" [exception_handling.InvalidAadharNumberException](#): Invalid Aadhar Number
at exception_handling.Aadhar.main([Aadhar.java:14](#))

Task 2 – InsufficientBallanceException

```
package exception_handling;
```

```
public class InsufficientBallanceException extends Exception {  
    public InsufficientBallanceException(String msg)  
    {  
        super(msg);  
    }  
}
```

```
package exception_handling;
```

```
import java.util.Scanner;
```

```
public class Bank {  
  
    public static void main(String[] args) throws InsufficientBallanceException {  
        double balance=10000;  
  
        Scanner sc=new Scanner(System.in);  
        System.out.println("Enter amount to withdraw: ");  
        double amount=sc.nextDouble();  
  
        if(amount>balance)  
        {  
            throw new InsufficientBallanceException("Insufficient Balance: "+ balance);  
        }  
        else  
        {  
            System.out.println("Withdrawal Successful: "+balance);  
        }  
    }  
}
```

Output –

Enter amount to withdraw:

12000

Exception in thread "main" [exception_handling.InsufficientBallanceException](#): Insufficient Balance: 10000.0
at [exception_handling.Bank.main\(Bank.java:16\)](#)

Difference Between throw & throws :-

Throw	Throws
Java throw keyword is used to explicitly throw an exception.	Java throws keyword is used to declare an exception.
Checked exception cannot be propagated using throw only.	Checked exception can be propagated with throws.
Throw is followed by an instance.	Throws is followed by class.
Throw is used within the method.	Throws is used with the method signature.
You cannot throw multiple exceptions.	You can declare multiple exceptions e.g. public void method()throws IOException,SQLException.